

SungEun Kim

DEVELOPER · BACKEND

Developer.

Silent Architect.

— 보이지 않는 곳에서 구조를 짓는 사람.



— — — PROFILE

Profile.

한 줄로 요약된 사람보다,
구조와 결로 설명되는 사람.

*A short record of
what I build with.*

01 Identity.

인 적 사 항

SungEun Kim
+82 10 · 6462 · 0602
kimsungeun0318@naver.com
Seoul, Korea

02 Education.

학 력

성결대학교
정보통신공학과
AI 풀스택 부트캠프 수료
2025.12 — 2026.06

03 Skills.

기 술 스 태 크

Back-End

Java · Spring Boot · Spring Data JPA · Node.js ·
RestAPI · JDBC · C

AI / Python

Python · FastAPI · LangChain · CNN · NLP

Front-End

React · JavaScript · Flutter · WebSocket · Axios

DevOps

AWS (EC2 / RDS / S3) · Git · GitHub Actions ·
JIRA

Database

MySQL · PostgreSQL · MongoDB · Redis · SQL
Tuning

Process

Plan → Design → Develop → Deploy → Google
Play Store

04 Certifications.

자 격 증

정보처리기사 · 2026 (필기)

05 Awards.

수 상 경 력

웹서버 프로그래밍 경진대회 · 2025.07 (교내)
ICT 통합 설계 경진대회 · 2025.12 (교내)

Projects.

세 개의 프로젝트,
각자의 문제와 해법.

*Three problems,
three quiet builds.*

01

NOOS.

2024.12 — 2026.06

뇌파 기반 인지·감성 판단 피드백 시스템 - Project:NOOS

React · Spring Boot · MyBatis · MySQL · FastAPI · Python · Expo · BLE · ACE-Step

02

Swimming Starter

2025.03 — 진행 중

수영 훈련 프로그램

Flutter · Dart · Spring Boot · JPA · QueryDSL · PostgreSQL · FCM · AWS S3

03

Groot.

2026.03 — 2026.04

탄소 흡수량 기반 ESG 플랫폼

React · TypeScript · Spring Boot · JPA · MySQL · AWS

Project No.01

뇌파 기반 인지 · 감성 판단 피드백 시스템

뇌파에서 감정의 결을 읽고,
사용자의 상태에 응답하는 시스템.

Project Overview

— CORE MISSION

사용자의 뇌파와 자기보고 응답을 읽어, 지금 상태에 맞는 음악과 조명을 자동으로 만들어 주는 적응형 몰입 환경 시스템.

개발 배경

디지털 과부하·집중력 저하·정서적 피로 / 자신의 상태를 실시간으로 인지하기
어려운 환경

해결하려는 문제

사용자 상태와 무관한 일방적 자극 / 현재 상태에 반응하는 환경 제어 구조 부재

주요 기능

뇌파 측정 → AI 상태 분석 → 목표에 맞춘 음악·조명 자동 생성 및 동기 제어

기대 효과

뇌파 반응형 몰입 환경 구현 / 사용 기록 누적 시 개인 맞춤·학습형 환경으로 확장

차별화 포인트

단일 입력 기반 기존 서비스(Endel·Neurocity) 대비 / 뇌파 + 사용자 응답 융합 / 음악·
조명 분리 검증

담당 역할

4인 팀 / 사용자 화면 흐름·모바일 확장 설계 / 백엔드·AI 팀과 화면-기능 연결 명세 정의
및 통합 테스트 협업

기술 스택 / 개발 환경

Java 17 · Spring Boot · MyBatis · MySQL / Python · FastAPI · NumPy(FFT) · ACE-Step / React 19 · Vite · Web Bluetooth / Jetson Nano · ESP32 · WiZ

Key Screens

홈 페이지 초기화면

Redefining focus, creativity, and balance in real time.

장인의 생각과 감정을 뛰어 넘어 최상의 몰입과 취미를
선사하는 지능형 공간.

Available on Web

메인 대시보드

LIGHTING PRESCRIPTION

BALANCED SUSTAIN LIGHTING

중간 색온도와 밝은 조도 조합으로 장시간 업무에서 편안함과 생산성 균형을 맞춥니다.

4200 K · 500 LX

직접 눈으로

Primary
#f4d5b1

Secondary
#8fe3c2

Accent
#8db5ff

TARGET CCT

4200 K

RESEARCH LUX

500 lx

BRIGHTNESS

58%

PATTERN

정적 유지

Device payload generic-rgb · primary RGB 244, 213, 177

Balanced Focus Field

세션 전반

4200 K · 500 lx · 58% · Balanced Hold

Chen et al. 2022 Bao 2021

TARGET STATE

사용자 설문조사 화면

ANALYSIS COMPLETE

정서 긴장 상태

AI Current State

긴장과 걱정 신호가 비교적 높아 안정화 처방이 우선인 상태입니다.

AI STATE INTERPRETATION

주의집중 15/20, 이완 9/15, STAI-6 17/24, 피로 10/20, KSS 5/9, 정신노력 5/9로 나타났습니다. 이를 AI 엔진 상태 벡터로 변환하면 집중 준비도 56/100, 스트레스 부하 57/100, 피로 위험 41/100, 이완 수준 47/100, 각성 수준 57/100, 정신 부하 55/100이며, 최종적으로 '정서 긴장 상태' 패턴으로 해석됩니다.

주의집중 15/20	이완 9/15
STAI-6 17/24	피로 10/20
KSS 5/9	정신노력 5/9

AI STATE VECTORS

검증 설문 항목 기반 장황 지표

집중 준비도	56/100
집중은 가능하지만 외부 자극이나 피로의 영향을 받을 수 있습니다.	확인
스트레스 부하	57/100
긴장 신호가 일부 존재하지만 통제 가능한 범위입니다.	확인
피로 위험	41/100
피로가 쌓이기 시작한 구간으로 보입니다.	확인
이완 수준	47/100
안정감은 있으나 긴장 완화 여지가 남아 있습니다.	확인
각성 수준	57/100

피드백 결과 화면

Device payload generic-rgb · primary RGB 255, 241, 232

Action Preset

4200 K · 1000 lx · 84% · Spot Ramp

이번 세션이 실제로 어땠는지 남겨주세요.

별점과 자연어 피드백을 함께 받으면 다음 음역/조향 생성에 직접 반영됩니다.

1 2 3 4 5

예: 조명이 조금 차갑고 음악이 살짝 긴장했어요. 집중은 잘 됐는데 오래 듣기엔 피곤했습니다.

GOAL 0% MUSIC 0% LIGHTING 0%

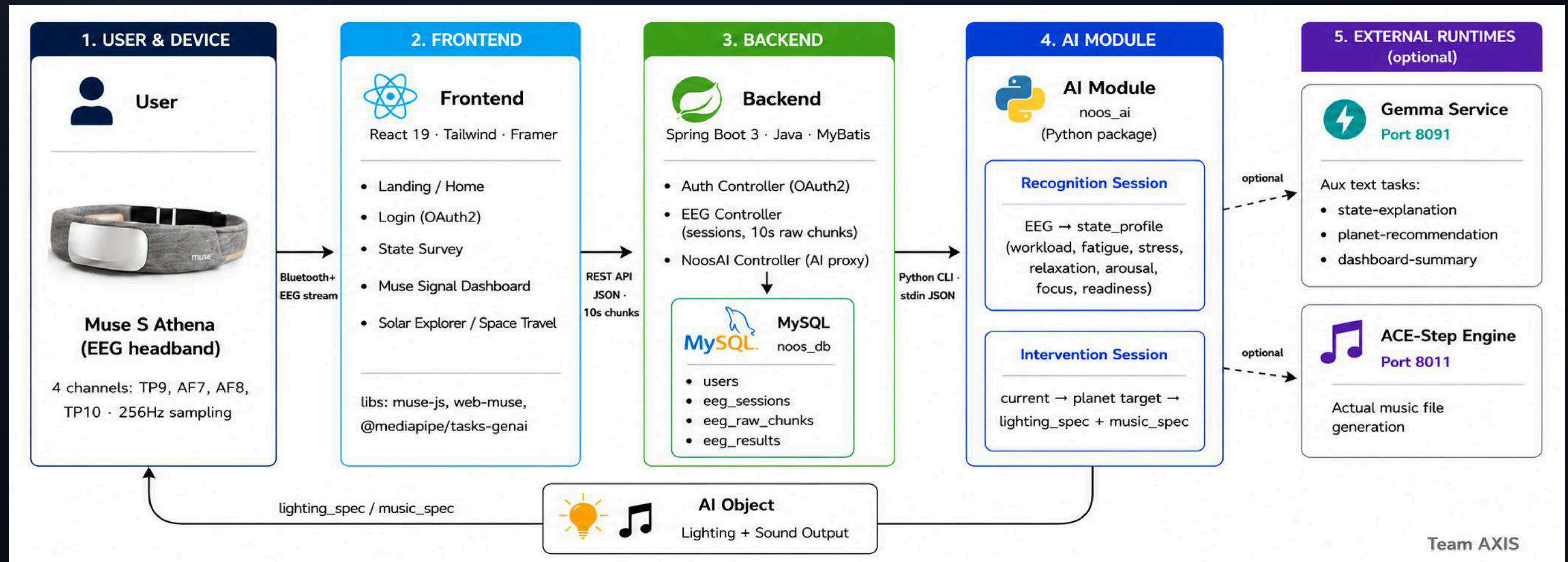
사용자는 현재 인지 부하 상태가 높고 피로 위험이 높은 상태에서 '절단과 실행'이라는 목표를 가지고 있습니다. 전반적인 상태는 집중력과 스트레스 부하가 높습니다.

AI 세션 미리보기

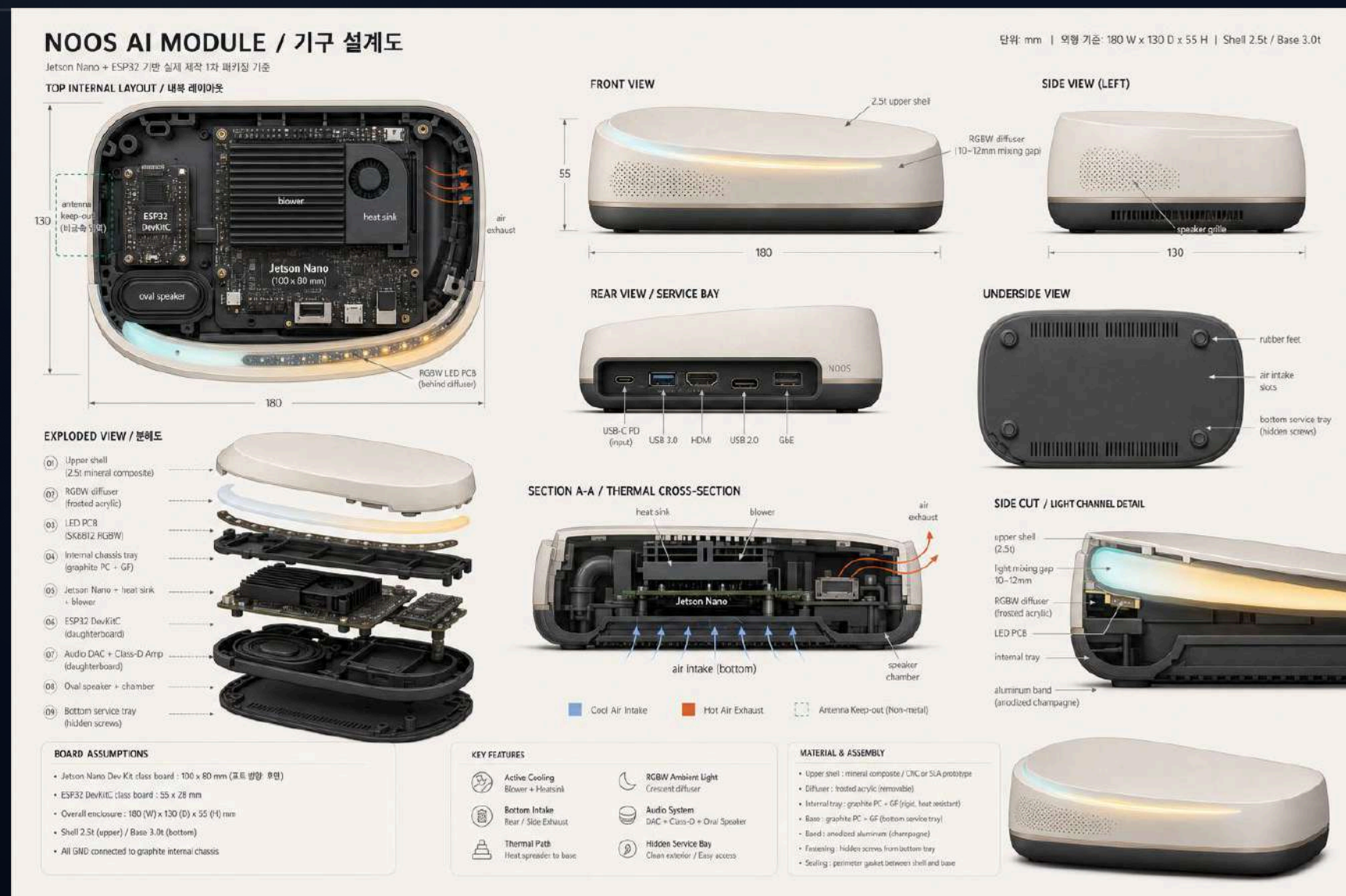
피드백 저장 후 이동

이전 세션에서 기록된 데이터와 현재 세션 데이터를 비교하여 분석하고, 최적의 결과를 도출합니다.

Architecture



Hardware Design & Layout

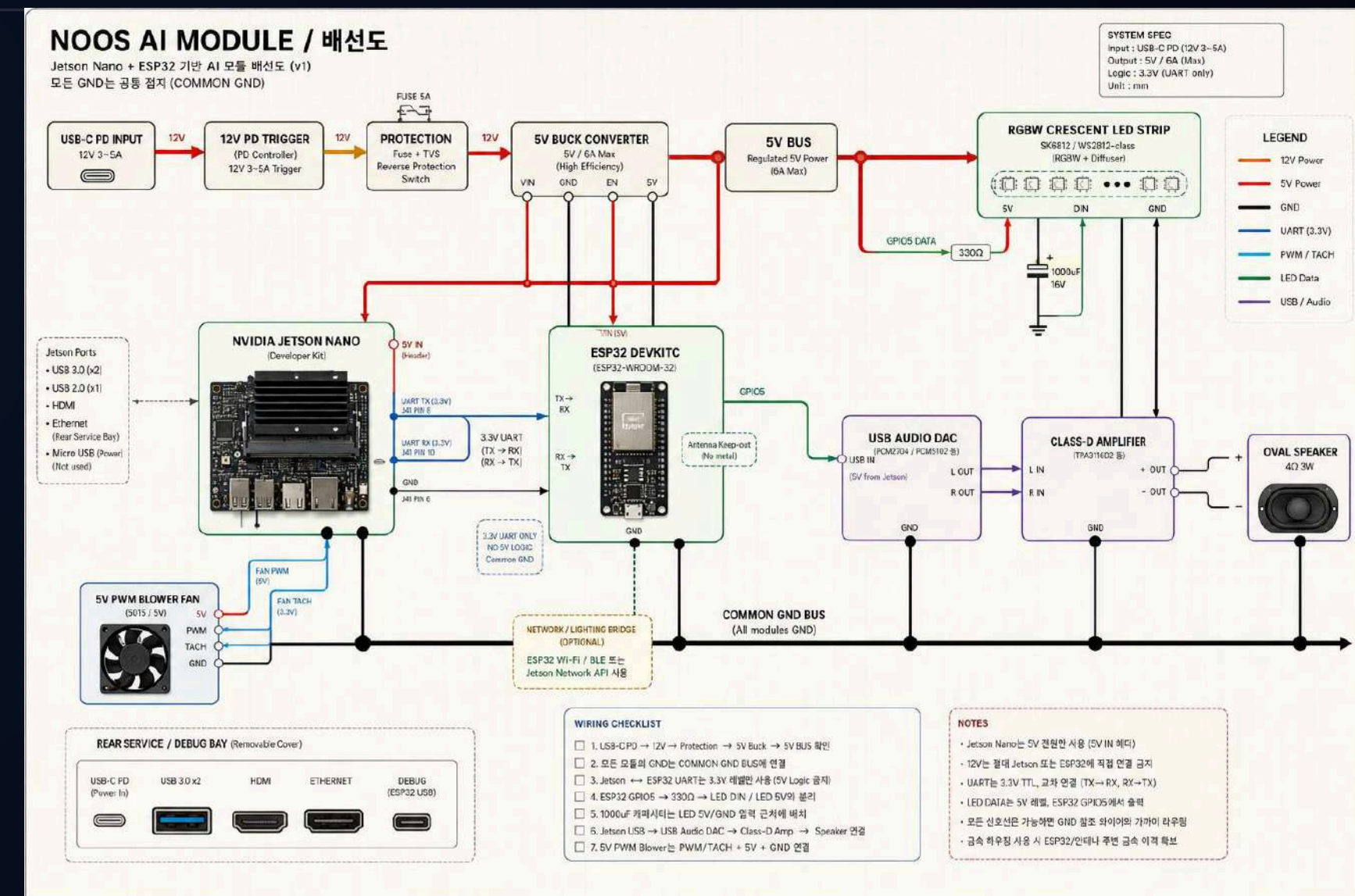


기구 설계

Jetson Nano와 ESP32를 실제 수납할 수 있는 180 x 130 x 55 mm 규모의 낮은 오브제형 본체 구조로 재정리.

부품 배치

Jetson Nano, 히트싱크, 블로워 팬, 흠배기 덕트, 하부 서비스 트레이를 중심으로 내부 레이아웃 구성.



전원 · 제어 · 배선

USB-C PD 입력을 12 V로 받고 5 V buck converter로 분배. Jetson Nano와 ESP32는 3.3 V UART로 제어 통신.

Core Logic

```
_infer_dimensions(features, quality_score, feature_source, has_baseline)
global_rel = features["global_relative"]
frontal_rel = features["regional_relative"]["frontal"]
ratios = features["ratios"]
hf_ratio = features["artifact_indicators"]["high_frequency_ratio"]

workload_score = clamp(
    0.55 * _scale(frontal_rel["theta"], 0.12, 0.24)
    + 0.15 * _scale(frontal_rel["beta"], 0.12, 0.24)
    + 0.15 * (1.0 - _scale(frontal_rel["alpha"], 0.18, 0.34))
    + 0.15 * _scale(ratios["frontal_theta_beta_ratio"], 0.8, 1.8)
    - 0.10 * _scale(ratios["alpha_beta_ratio"], 1.8, 2.8),
    0.0,
    1.0,
)

fatigue_score = clamp(
    0.30 * _scale(global_rel["theta"], 0.14, 0.28)
    + 0.25 * _scale(global_rel["alpha"], 0.20, 0.40)
    + 0.25 * _scale(ratios["alpha_beta_ratio"], 1.2, 2.8)
    + 0.20 * (1.0 - _scale(global_rel["beta"], 0.14, 0.26))
    - 0.10 * _scale(frontal_rel["beta"], 0.18, 0.28),
    0.0,
    1.0,
)

stress_score = clamp(
    0.40 * (1.0 - _scale(global_rel["alpha"], 0.18, 0.34))
    + 0.30 * _scale(global_rel["beta"], 0.14, 0.28)
    + 0.20 * _scale(ratios["beta_alpha_ratio"], 0.7, 1.6)
    + 0.10 * _scale(frontal_rel["beta"], 0.14, 0.26),
    0.0,
    1.0,
)

focus_readiness_score = clamp(
    0.30 * (1.0 - fatigue_score)
    + 0.25 * (1.0 - stress_score)
    + 0.25 * (1.0 - clamp(abs(workload_score - 0.45) / 0.45, 0.0, 1.0))
    + 0.20 * quality_score,
    0.0,
    1.0,
)

return {
    "mental_workload": build_dimension(workload_score, quality_score, feature_source, has_baseline),
    "fatigue_risk": build_dimension(fatigue_score, quality_score, feature_source, has_baseline),
    "stress_load": build_dimension(stress_score, quality_score, feature_source, has_baseline),
    "focus_readiness": build_dimension(focus_readiness_score, quality_score, feature_source, has_baseline)
}
```

핵심 코드

CURRENT STATE SNAPSHOT

Session vectors

핵심 화면

재생 직전 기준으로 NOOS가 읽은 핵심 축을 정리한 상태 카드입니다.

FOCUS READINESS

44%

몰입으로 진입할 준비 정도

STRESS LOAD

56%

우선 완화가 필요한 긴장량

FATIGUE RISK

53%

세션 지속을 방해할 수 있는 피로 신호

AI INTERVENTION

Planetary preset

아직 AI 생성 결과가 없어서 행성 기본 처방을 재생하고 있습니다.

TRANSITION INTENSITY

0%

상태를 얼마나 크게 움직이려는지에 대한 개입 강도

RELIABILITY

0%

현재 입력 품질과 상태 해석을 바탕으로 계산한 신뢰도

INPUT QUALITY

0%

측정 또는 설문 입력의 안정성 수준

LIGHTING PRESCRIPTION

BALANCED SUSTAIN LIGHTING

중간 색온도와 밝은 조도 조합으로 장시간 업무에서 편안함과 생산성 균형을 맞춥니다.

4200 K - 500 LX

적정 눈문길

Primary #f4d5b1

Secondary #8fe3c2

Accent #8db5ff

TARGET CCT

4200 K

RESEARCH LUX

500 lx

BRIGHTNESS

58%

PATTERN

정적 유지

Device payload generic-rgb - primary RGB 244, 213, 177

Balanced Focus Field

4200 K · 500 lx · 58% · Balanced Hold

Chen et al. 2022 Bao 2021

TARGET STATE

균형형 집중

과하게 긴장하지 않으면서도 안정적으로 집중을 유지하도록 설계된 환경입니다. 장시간 업무, 학습, 루틴 작업을 편안하게 이어가고 싶을 때 적합합니다.

FOCUS READINESS

44%

RELAXATION

22%

AROUSAL

22%

CORE LOGIC

EEG 기반 인지 상태 분석

EEG 신호의 theta, alpha, beta 비율을 정규화해 집중 준비도, 스트레스 부하, 피로 위험, 인지 작업부하를 계산하는 규칙 기반 인지 상태 추론 로직 구현함.

여러 band power와 ratio를 함께 사용해 단일 지표가 아닌 복합 상태로 사용자의 현재 인지 상태 판단.

SCREEN RESULT

상태 카드와 몰입 세션 추천

분석 결과를 Focus readiness, Stress load, Fatigue risk 상태 카드로 시각화함.

사용자가 현재 집중 가능한 상태인지, 완화가 필요한지, 회복이 필요한지 확인하고 맞춤형 몰입·회복 세션 추천으로 이어짐.

Troubleshooting

```
import { MuseClient } from 'muse-js';

const CHANNEL_MAP = {
  0: 'tp9',
  1: 'af7',
  2: 'af8',
  3: 'tp10',
  4: 'aux',
};

async function startMuseStream() {
  const muse = new MuseClient();

  await muse.connect();
  await muse.start();

  const eegBuffer = {
    tp9: [],
    af7: [],
    af8: [],
    tp10: [],
    aux: [],
  };

  muse.eegReadings.subscribe((reading) => {
    const channel = CHANNEL_MAP[reading.electrode];

    if (!channel) return;

    eegBuffer[channel].push(...reading.samples);

    updateRawEegState({
      source: 'muse-sdk',
      channels: eegBuffer,
      connected: true,
    });
  });
}
```

Before 코드



TROUBLESHOOTING

구형 EEG SDK 호환성 문제를 자체 매핑 구조로 해결

Before 코드의 SDK 의존 문제를 앞선 핵심 코드 화면의 정규화·가중치 계산 구조로 전환함.

PROBLEM

문제

초기에는 Muse SDK로 5개 EEG 채널의 raw signal을 수집하려 했으나, SDK가 오래되어 최신 Web Bluetooth 환경에서 연결 안정성이 낮았음. 또한 raw EEG를 집중도·스트레스·피로 지표로 바로 활용하기 어려웠음.

SOLUTION

해결

SDK 의존도를 낮추고, 수집 가능한 EEG feature를 앱 상태 모델에 맞게 직접 정규화·매핑하는 구조로 전환함. 앞선 핵심 코드 화면의 정규화·가중치 계산 로직을 기반으로 theta, alpha, beta band와 주요 ratio를 조합함.

RESULT

결과

EEG 신호를 집중 준비도, 스트레스 부하, 피로 위험, 인지 작업부하로 변환할 수 있었음. 계산된 상태값을 상태 카드 UI와 맞춤형 몰입·회복 세션 추천 로직에 연결함.

Retrospective

KEEP

잘 된 것 · 계속

- ✓ EEG 데이터를 단순 시각화에서 끝내지 않고, 서비스에서 활용 가능한 인지 상태 지표로 변환
- ✓ theta, alpha, beta band와 주요 ratio를 기반으로 집중 준비도, 스트레스 부하, 피로 위험, 인지 작업부하 계산
- ✓ 분석 결과를 상태 카드 UI와 맞춤형 몰입·회복 세션 추천까지 연결

PROBLEM

아쉬운 것 · 개선 대상

- × 초기에는 Muse SDK로 raw EEG 데이터를 수집하려 했으나, 최신 Web Bluetooth 환경에서 연결 안정성이 낮았음
- × raw signal을 그대로 사용하는 구조로는 집중도·스트레스·피로 지표로 바로 활용하기 어려웠음
- × 사용자별 baseline과 신호 품질 차이를 충분히 반영하지 못한 한계가 남음

TRY

다음에 시도할 것

- 실제 사용자 측정 데이터를 더 많이 수집해 기준값을 보정
- 개인별 baseline을 반영한 scoring 로직으로 개선
- 신호 품질이 낮은 구간을 자동 제외하고, 신뢰도 점수를 더 정확하게 계산해 분석 안정성 향상

— PROJECT 02

Project *Swimming Starter*

수영 훈련 프로그램

수영을 시작하는 사람을 위한,
훈련의 첫 흐름을 담은 프로그램.

— — — 2 - 1 OVERVIEW

Project Overview

— CORE MISSION

수영 훈련의 처음부터 끝까지, 한 화면 안에서.

개발 배경

전(前) 수영선수 출신 의뢰인이 현장에서 겪은 운영 불편을 한 화면에 통합하기 위해 시작된 프로젝트.

기대 효과

하드웨어 의존 제거, 세션 준비 시간 단축. 수영용품 광고 · 쇼핑물 연계까지 확장 가능.

기술 스택 / 개발 환경

Flutter 3.7 · Dart · Riverpod / Spring Boot · Java 21 · JPA · QueryDSL · JWT / PostgreSQL · AWS EC2 · RDS · S3 / GitHub Actions · RevenueCat (월/연 구독)

주요 기능

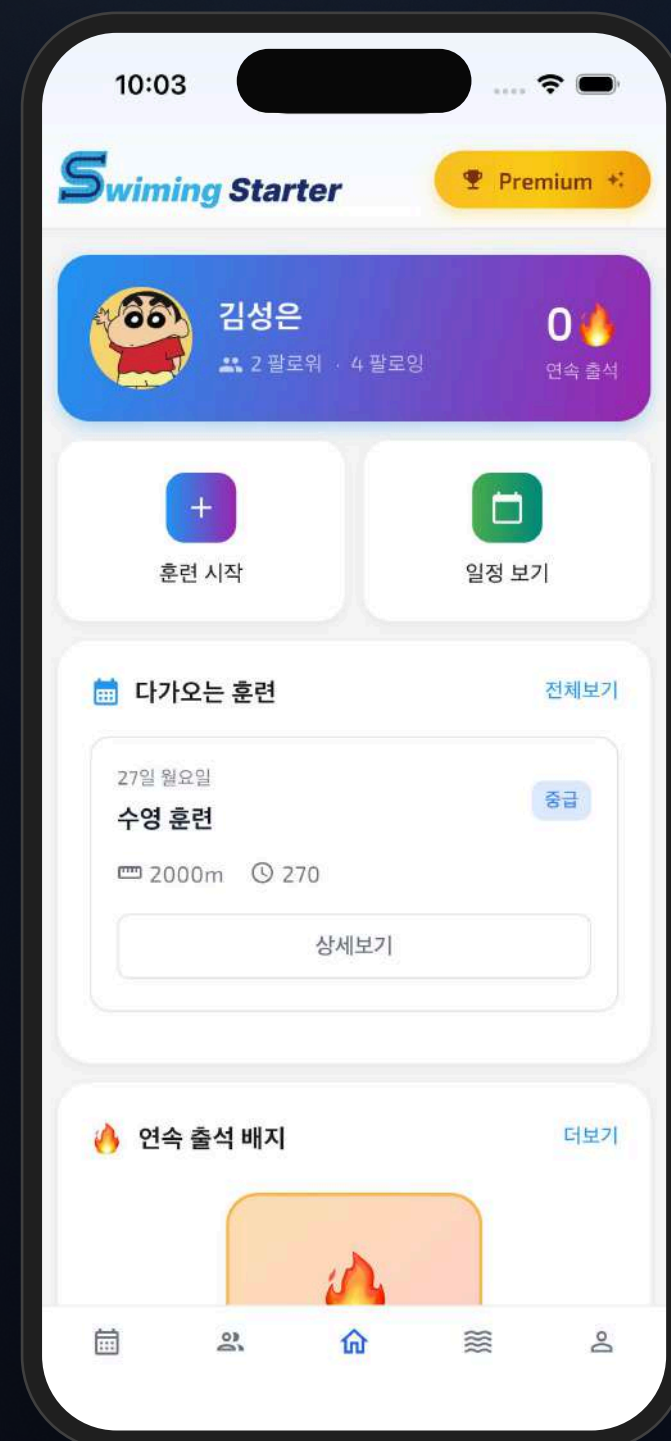
커스텀 훈련 빌더 · 다중 출발 인터벌 · 퀵스타트 · 캘린더 자동 기록 · 훈련 공유 커뮤니티.

차별화 포인트

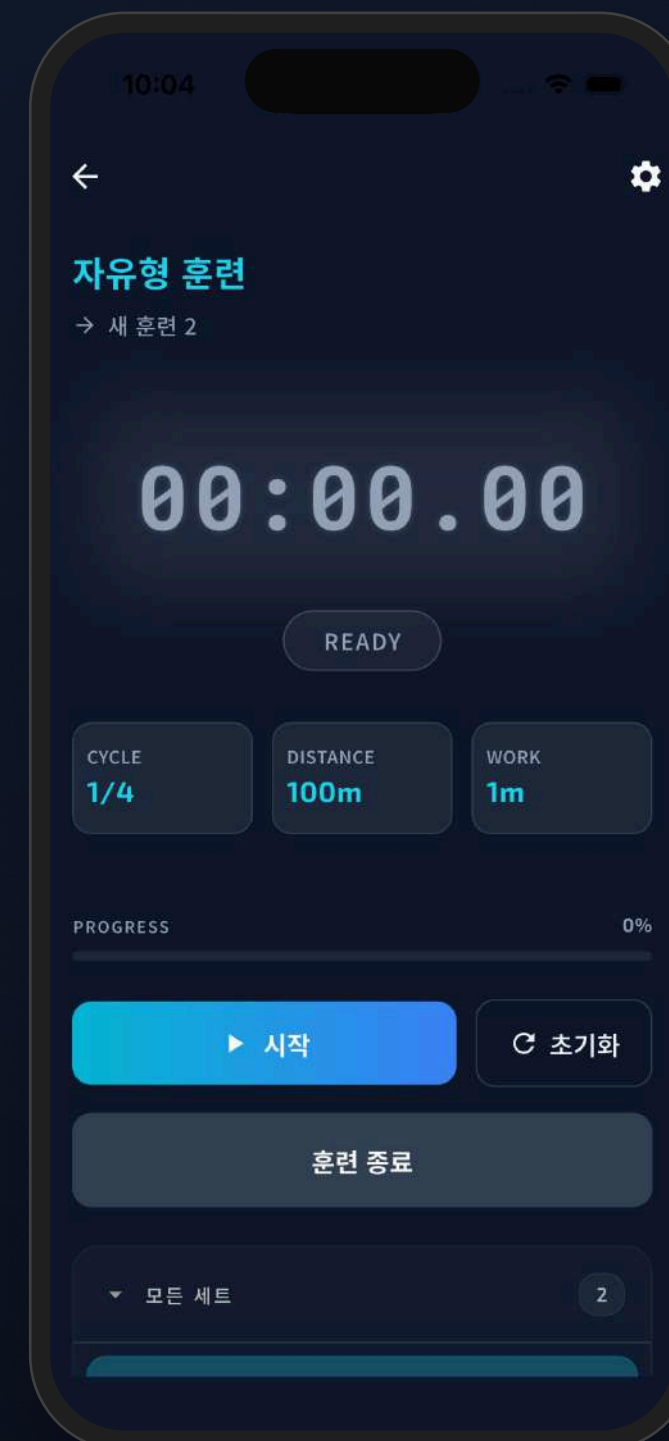
기존 앱은 개인 단일 타이머 수준. 다중 출발 인터벌·퀵스타트·커뮤니티·캘린더를 한 앱에 통합한 점이 결정적인 차이.



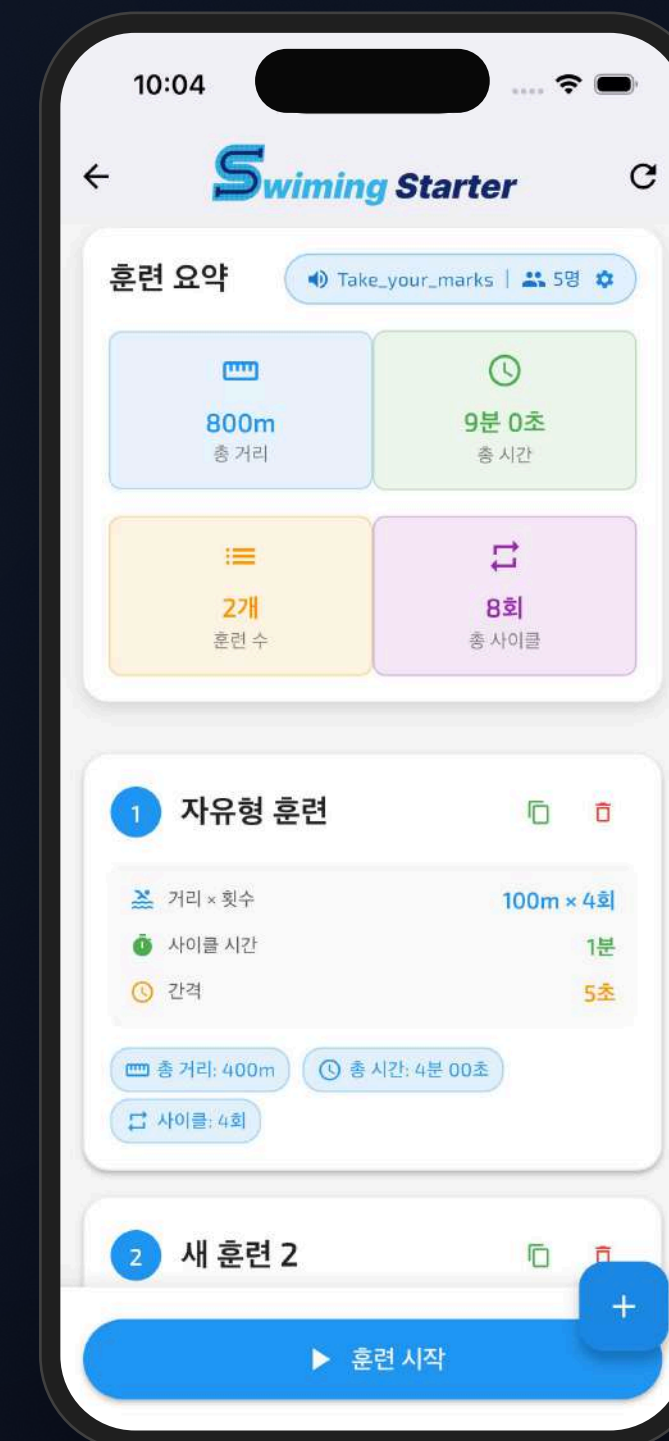
Key Screens



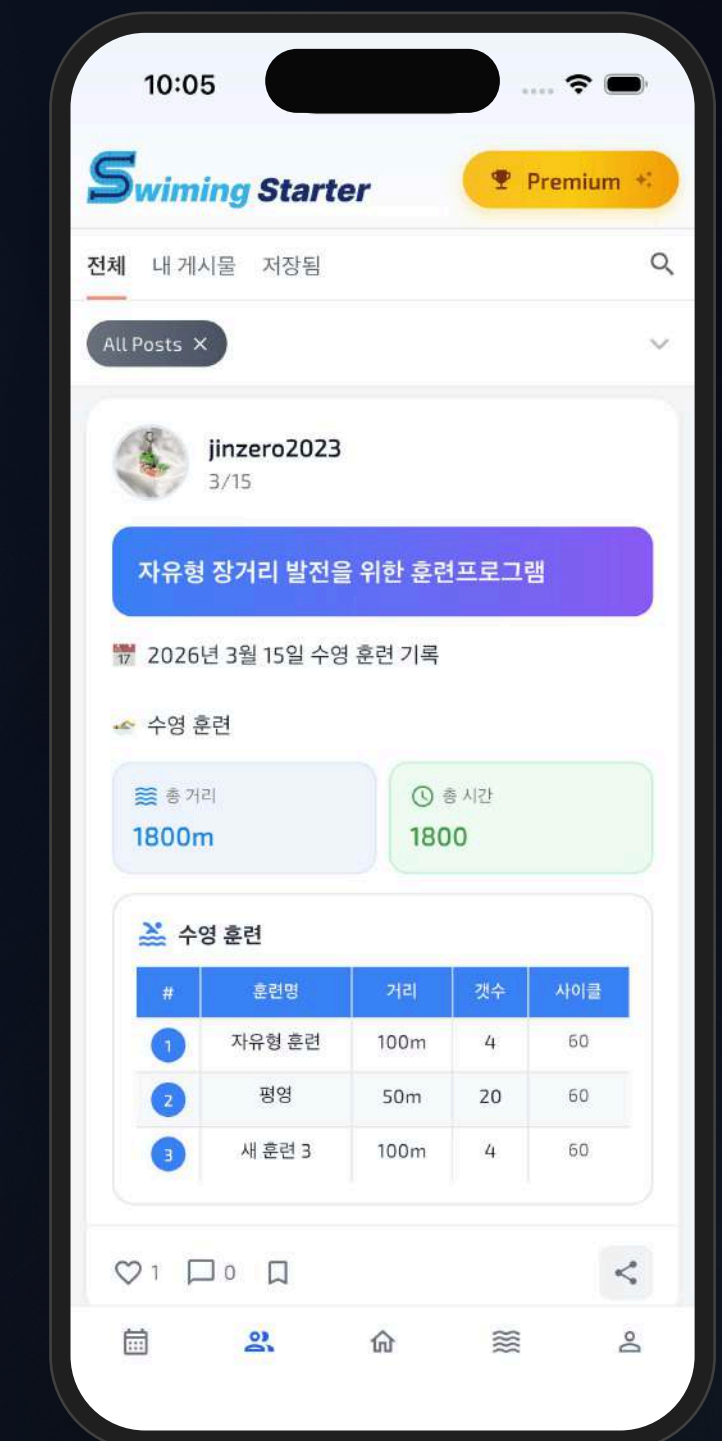
커뮤니티 화면



훈련 타이머 화면

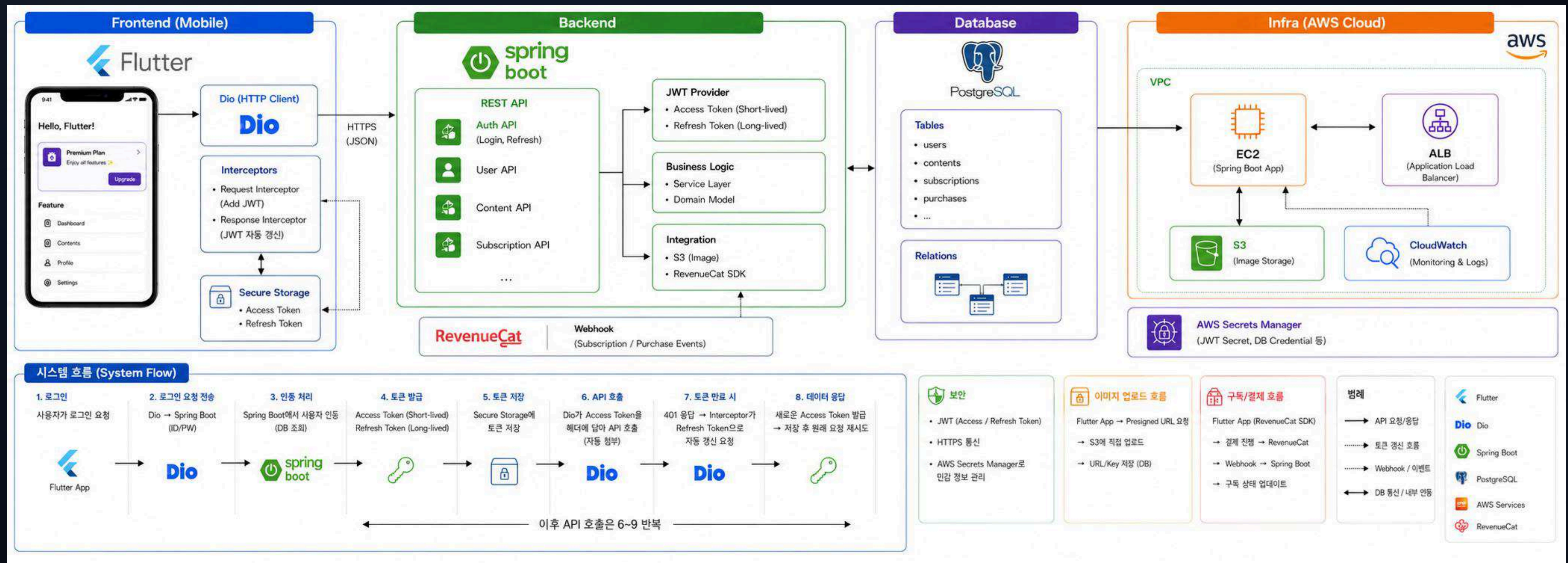


훈련 작성 화면



메인 홈

Architecture



Development Workflow



```

trees/.../manage-skills/SKILL.md  M+ .claude/.../manage-skills/SKILL.md x  M+ skills/verify-deploy/SKILL.md

name: manage-skills
description: 세션 변경사항을 분석하여 검증 스킬 누락을 탐지합니다. 기존 스킬을 동적으로 탐색하고, 새 스킬을 생성하고
disable-model-invocation: true
argument-hint: "[선택사항: 특정 스킬 이름 또는 집중할 영역]"
---
# 세션 기반 스킬 유지보수

## 목적
현재 세션에서 변경된 내용을 분석하여 검증 스킬의 드리프트를 탐지하고 수정합니다:

1. **커버리지 누락** - 어떤 verify 스킬에서도 참조하지 않는 변경된 파일
2. **유효하지 않은 참조** - 삭제되거나 이동된 파일을 참조하는 스킬
3. **누락된 검사** - 기존 검사에서 다루지 않는 새로운 패턴/규칙
4. **오래된 값** - 더 이상 일치하지 않는 설정값 또는 탐지 명령어

## 실행 시점
- 새로운 패턴이나 규칙을 도입하는 기능을 구현한 후
- 기존 verify 스킬을 수정하고 일관성을 점검하고 싶을 때
- PR 전에 verify 스킬이 변경된 영역을 커버하는지 확인할 때
- 검증 실행 시 예상했던 이슈를 놓쳤을 때
- 주기적으로 스킬을 코드베이스 변화에 맞춰 정렬할 때

## 등록된 검증 스킬
현재 프로젝트에 등록된 검증 스킬 목록입니다. 새 스킬 생성/삭제 시 이 목록을 업데이트합니다.

| 스킬 | 설명 | 커버 파일 패턴 |
|-----|-----|-----|
| `verify-build` | Flutter analyze + Backend 컴파일 검증 | `pubspec.yaml`, `analysis_options`
| `verify-api-sync` | 프론트/백엔드 API 엔드포인트 일관성 검증 | `lib/core/api/services/*_api_serv`
| `verify-auth` | JWT 인증 플로우 전체 검증 | `lib/core/auth/*.dart`, `lib/core/api/api_client`
| `verify-deploy` | EC2 배포 전 사전 검증 | `swim-backend/build.gradle.kts`, `swim-backend/
  
```


— — — 2 - 5 CASE STUDY

Case Study

REAL CLIENT RESPONSE

Quick Start with Quest System — actual case from production

REQUEST

Client message

손 혼드는 스카피

스위밍스타터
퀵스타트
정해져 있는 목록
1주차 훈련
2주차 훈련
3주차 훈련
4주차 훈련
퀘스트 깨기!(퀘스트를 깨야 다음 레벨
진입 가능)
각 훈련별 마지막 단계는 결제 후 이용
가능

광고
수영 용품 광고 홈페이지 연결

오후 7:05

손 혼드는 스카피

제목을 입력해주세요_.pdf
유효기간 ~2025. 9. 25.
용량 19.67 MB

퀘스트 화면 추가 된 이미지 보내드려요

광고 화면이랑 튜토리얼 화면 만드는
대로 다시 보내드릴게요

오후 8:19

PLAN

SPEC.md authored

```
./demo.sh
Claude Code session
• reading KAKAO_REQUEST.txt
• cross-referencing CLAUDE.md operating rules
• drafting Quick Start spec (scope · screens · risks)
/ SPEC.md written (94 lines)

EC.md (excerpt)

# Quick Start with Quest System - SPEC

> Source: KakaoTalk request from PM (손 혼드는 스카피)
> Attached design: 제목을 입력해주세요_.pdf
> Authored by Claude before implementation (pre-validation gate).

## 1. Scope

Add a new "Quick Start" entry point for new users that guides them through
4 weekly training tracks, gated by a per-track Quest and an in-app paywall
on the final step.

## 2. Screens

| Screen | Purpose |
| --- | --- |
| QuickStartScreen | Hero card + 4 weekly track cards + bottom ad slot |
| TrackDetailScreen | Step list (lessons + final paywall step) + Quest CTA |
| QuestCompleteSheet | Modal confirming quest cleared + unlocks next track |
| AdRedirectHandler | 'url_launcher' to swimwear homepage (externalApplicat...
```

CODE

Implementation

```
STEP 3 · CODE (Codex implements per phase plan)

$ git log --oneline
8d2f88bc feat(quickstart): add QuickStartScreen with 4 weekly tracks + ad red
irect
61a9e893 feat(quickstart): add QuestService with payment-gate unlock
24d14d71 feat(quickstart): add Quest + Track data models
9956a921 docs: capture client request + SPEC.md for Quick Start with Quest Sy
stem
28e9310e Update frontend release notes
caf61164 Migrate app features to backend services

$ git diff --stat HEAD~4..HEAD
KAKAO_REQUEST.txt | 22 ++++++
SPEC.md | 64 ++++++
lib/features/quickstart/models/quest.dart | 34 ++++++
lib/features/quickstart/models/track.dart | 29 ++++++
.../quickstart/screens/quick_start_screen.dart | 65 ++++++
.../quickstart/services/quest_service.dart | 35 ++++++
lib/features/quickstart/widgets/track_card.dart | 56 ++++++
7 files changed, 305 insertions(+)

Sample file (lib/features/quickstart/services/quest_service.dart)

import 'package:flutter/foundation.dart';

import '../core/subscription/subscription_service.dart';
import '../models/quest.dart';
import '../models/track.dart';

/// Quest unlock + payment gate orchestrator.
///
/// CLAUDE.md operating rules respected:
/// - premium check via 'SubscriptionService.instance.isPremium'
/// - never touch Firebase
/// - Spring page params 0-indexed
class QuestService {
  QuestService._();
  static final QuestService instance = QuestService._();

  Future<bool> canEnter(Track track) async {
    if (track.week == 1) return true;
    return track.quest.completed;
  }
}
```

VERIFY

verify-build

```
./demo.sh

✓ STEP 4 · VERIFY (verify-* skills run automatically)

$ claude /verify-build

Running Flutter analyze ...
✓ flutter analyze - No issues found
Running Spring Boot compile ...
✓ ./gradlew compileJava - BUILD SUCCESSFUL

verify-build : PASS

$ claude /verify-api-sync
Checking Flutter calls → Spring controllers ...
✓ /api/v1/tracks → TracksController match
✓ /api/v1/quests/{id}/complete → QuestController match
✓ /api/v1/payment/check → PaymentController match

verify-api-sync : PASS (12/12 endpoints aligned)

$ claude /verify-auth
JWT refresh flow + premium gate ...
✓ 403 empty-body refresh path covered
✓ SubscriptionService.isPremium wired on paywall step

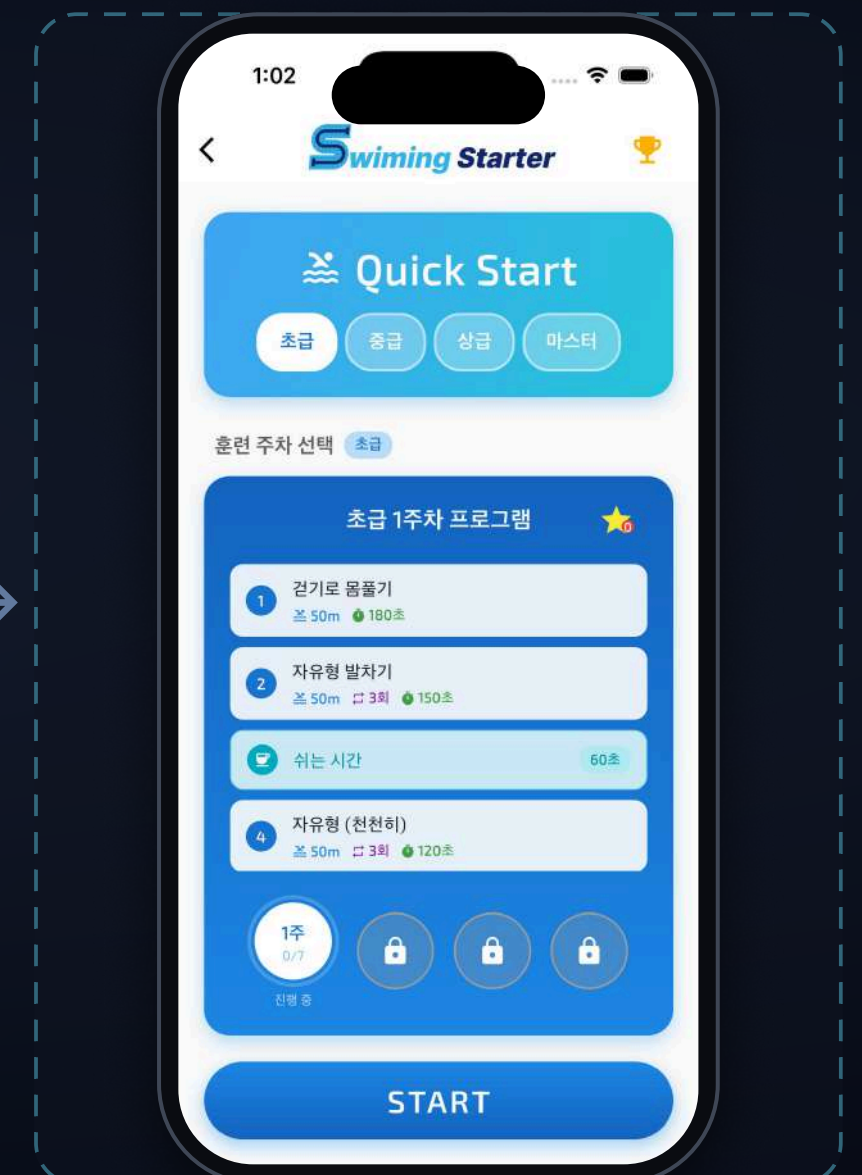
verify-auth : PASS

$ claude /verify-deploy
Pre-deploy checks ...
✓ all gates passed · ready for boot.sh → scp → ssh

All verify-* gates PASS · ready to deploy
```

DEPLOY

Live in app



Troubleshooting

✂ BEFORE · 스킬 없이 매 항목을 따로 물어볼 때

매 Phase 끝에서 검증 항목 하나하나를 따로 요청 → 누락·반복·일관성 부족

> 지금 빌드 잘 되는지 한번 확인해줘

```
$ flutter analyze
Analyzing swim...
No issues found! (3.1s)
✓ Flutter 빌드 통과했습니다.
```

> 백엔드도 컴파일 잘 되나?

```
$ cd ../swim-backend && ./gradlew compileJava
> Task :compileJava
BUILD SUCCESSFUL in 12s
✓ Spring 도 통과.
```

> Flutter에서 부르는 API랑 Spring 컨트롤러 매핑 일치해?

```
$ grep -roh '/api/v1/[a-z/{_}]*' lib/ | sort -u | wc -l
24
\033[2m백엔드 컨트롤러 4개 1:1 대조 중...\033[0m
✓ (직접 확인했지만 자동 기록은 없음)
```

> JWT 403 자동 갱신 동작도 확인해줘

```
\033[2mapi_client.dart 의 _AuthInterceptor 확인...\033[0m
\033[2m403 + empty body 분기 → token refresh 호출 OK\033[0m
✓ 정상 동작 - 단, 다음에 또 직접 물어봐야 함.
```

> 결제 게이트(isPremium) 연결됐는지도 봐줘

```
\033[2mSubscriptionService.instance.isPremium 호출처 검색...\033[0m
\033[2mquest_service.dart paypal step → isPremium 호출 확인\033[0m
✓ 연결됨.
```



동일한 절차를 슬래시 명령으로 묶어두고, Phase 종료 시 자동 호출

> /verify-build

```
$ flutter analyze
\033[0;32m\033[0m No issues
$ ./gradlew compileJava
\033[0;32m\033[0m BUILD SUCCESSFUL
verify-build : PASS
```

> /verify-api-sync

```
\033[0;32m\033[0m /api/v1/auth/refresh → AuthController
match
\033[0;32m\033[0m /api/v1/community/posts → CommunityController
match
\033[0;32m\033[0m /api/v1/community/posts/{id} → CommunityController
match
\033[0;32m\033[0m /api/v1/trainings → TrainingController
match
\033[0;32m\033[0m /api/v1/trainings/{id}/complete → TrainingController
match
\033[0;32m\033[0m /api/v1/users/me → UserController
match
verify-api-sync : PASS (24/24 endpoints aligned)
```

> /verify-auth

```
\033[0;32m\033[0m 403 empty-body refresh path covered
\033[0;32m\033[0m SubscriptionService.isPremium gate wired on paypal
\033[0;32m\033[0m TokenInterceptor 자동 재시도 동작
verify-auth : PASS
```

> /verify-deploy

```
\033[0;32m\033[0m build artifact present (swim-api-1.0.0-SNAPSHOT.jar)
\033[0;32m\033[0m boot.sh → scp → ssh pipeline ready
```

BEFORE · Manual verification

- × 검증 4종을 매번 따로 요청 — 누락 위험
- × 같은 일을 매 Phase 마다 반복 — ~3분 소요
- × 출력 형식 비일관 — PASS/FAIL 자동 기록 안 됨



✨ AFTER · verify-* skills

- ✓ 한 줄의 슬래시 명령으로 동일 절차 캡슐화
- ✓ 4개 합쳐 ~25초 (수동의 약 1/7)
- ✓ PASS/FAIL 자동 기록 · 형식 일관 · Phase 종료 시 자동 호출

Retrospective

“AI 자체가 아니라, AI를 운영하는 시스템이 품질을 결정했다.”

KEEP

잘 된 것 · 계속

- ✓ SPEC.md를 먼저 작성해 CLAUDE.md 규칙을 사전 반영, 재 시도 1회 이하로 감소
- ✓ verify-* 스킴로 빌드·API·인증·배포 검증을 한 줄 슬래시 명령에 묶어 자동화
- ✓ 반복 함정은 CLAUDE.md 영구 규칙으로 정리해 같은 실수 재발 방지

PROBLEM

아쉬운 것 · 개선 대상

- × SPEC 없이 요청하면 CLAUDE.md 규칙이 무시돼 withOpacity, page=1 같은 위반 재발
- × Agent Team 4명을 돌리면 단일 세션 대비 토큰이 3-4배 증가해 일상 작업에는 과함
- × 캡처·CHANGELOG·진행 기록은 여전히 사람 손에 의존해 트래킹 비용 발생

TRY

다음에 시도할 것

- Hooks PostToolUse로 파일 저장 시 verify-build 자동 발화
- Notion(일지 자동 기록), Linear(이슈), GitHub PR(설명 자동) MCP 추가 연결
- 카톡 의뢰 → SPEC.md → 코드 → verify → PR까지 풀체인 자동화

— — — PROJECT 03

Project Groot

탄소 흡수량 기반 ESG 플랫폼

탄소 흡수량을 측정하고 가시화하는,
지속 가능성을 위한 ESG 플랫폼.

Project Overview

— CORE MISSION

기업의 나무 심기 활동을 산림과학원 공인 계수 기반으로 정량 측정·인증하는 ESG 탄소중립 플랫폼.

개발 배경

탄소중립·ESG 확산에도 기업 식재 활동의 정량 인증 수단 부재. 수종별 흡수량 차이가 최대 수십 배에 달함.

핵심 목표

현장 실측(DBH·수고·좌표) + 논문 기반 상대생장식으로 탄소흡수량 자동 산출

주요 기능

답사 신청·전문가 자동배정·탄소 예측·인증마크 4등급·ESG 리포트

기대 효과

기업별 누적 데이터를 축적해 장기 탄소중립 로드맵을 지원하며, 향후 수종 DB 확대·탄소 크레딧 거래·정부 인증 연계로 확장 가능.

차별화 포인트

국립산림과학원 공인 계수 + 수령·계절·기온·날씨 4중 보정 알고리즘
→ 나무 추천 시스템

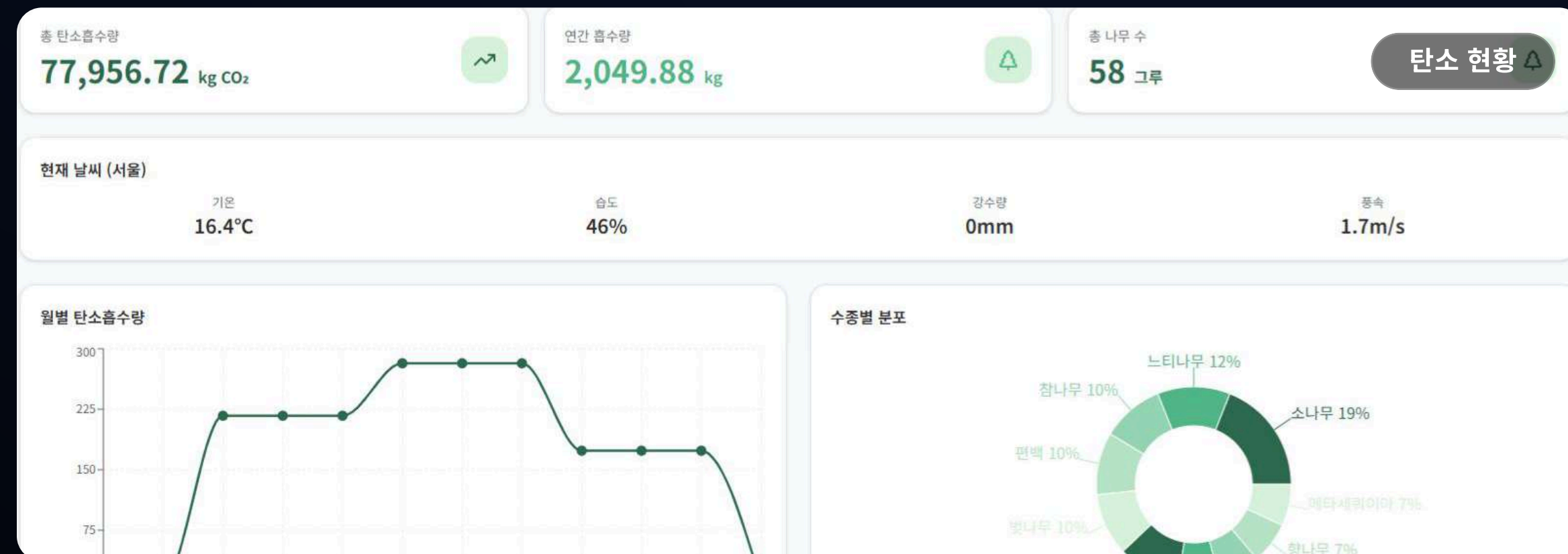
담당 역할

탄소 계산 엔진 / 나무 추천 알고리즘
/ 대시보드 / 인증마크 / 외부 API 4종 연동

기술 스택 / 개발 환경

Java 17 · Spring Boot 4.0.6 · JPA · MySQL · JWT · React · TypeScript · AWS(EC2/S3/RDS) · KOSIS · 기상청 · 농촌진흥청

Key Screens



수목 추천

지역 기후·토양 기반으로 최적 수종을 추천받아보세요.

기본 정보 — 2 수종 추천 결과

서울 면적: 2,000 m² 면적 기준 최대 식재: 222그루

추천 수종 TOP 3

1 신갈나무

낙엽활엽수

★ 91.5 / 100점

그루당 연간 흡수 **7.6 kg**

연간 총 흡수량 **229.06 kg**

식재 간격 **5.0 m**

배수조건 적합. 토심 적합. 기온 최적 구간. 예상 연간 흡수량 10.5kg/그루.

탄소흡수량 (35%)	93
토양적합도 (30%)	97
기후적합도 (20%)	75
면적적합도 (15%)	100

2 아까시나무

낙엽활엽수

★ 89.5 / 100점

그루당 연간 흡수 **8.1 kg**

연간 총 흡수량 **243.7 kg**

식재 간격 **3.0 m**

배수조건 적합. 토심 적합. 기온 최적 구간. 예상 연간 흡수량 11.2kg/그루.

탄소흡수량 (35%)	100
토양적합도 (30%)	97
기후적합도 (20%)	75
면적적합도 (15%)	70

3 상수리나무

낙엽활엽수

★ 82.9 / 100점

그루당 연간 흡수 **6.0 kg**

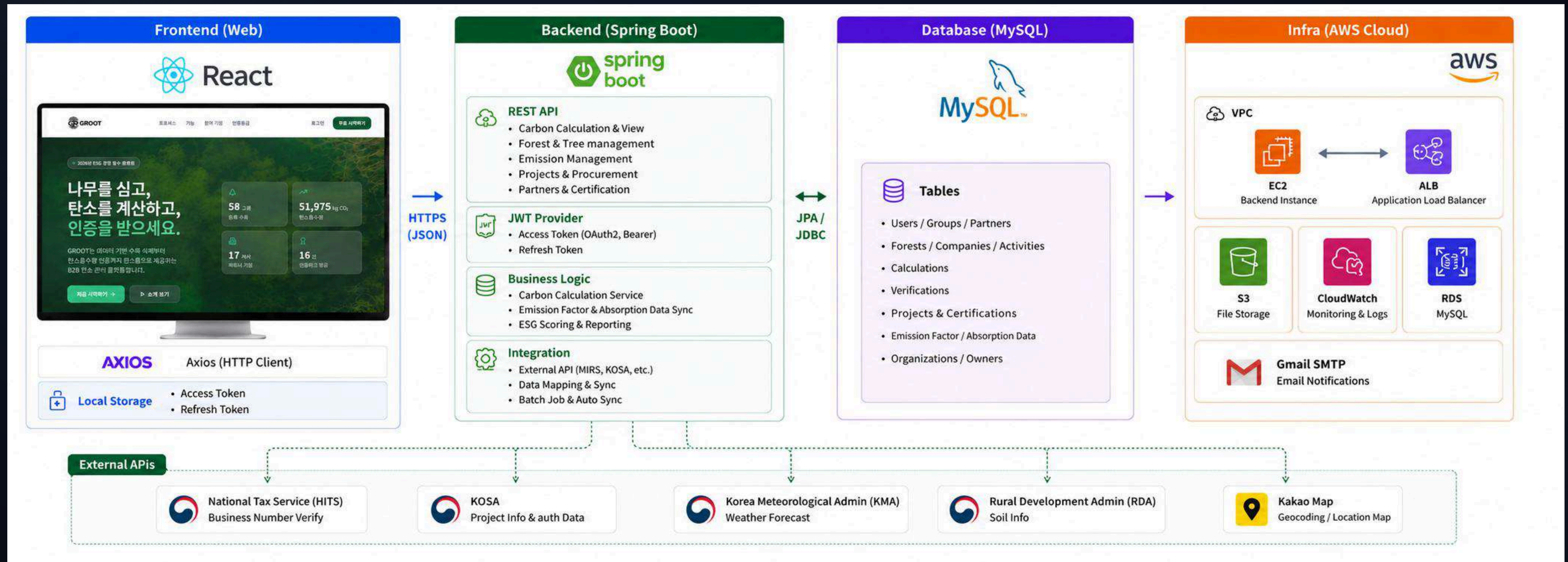
연간 총 흡수량 **178.84 kg**

식재 간격 **5.0 m**

배수조건 적합. 토심 적합. 기온 최적 구간. 예상 연간 흡수량 8.2kg/그루.

탄소흡수량 (35%)	68
토양적합도 (30%)	97
기후적합도 (20%)	75
면적적합도 (15%)	100

Architecture

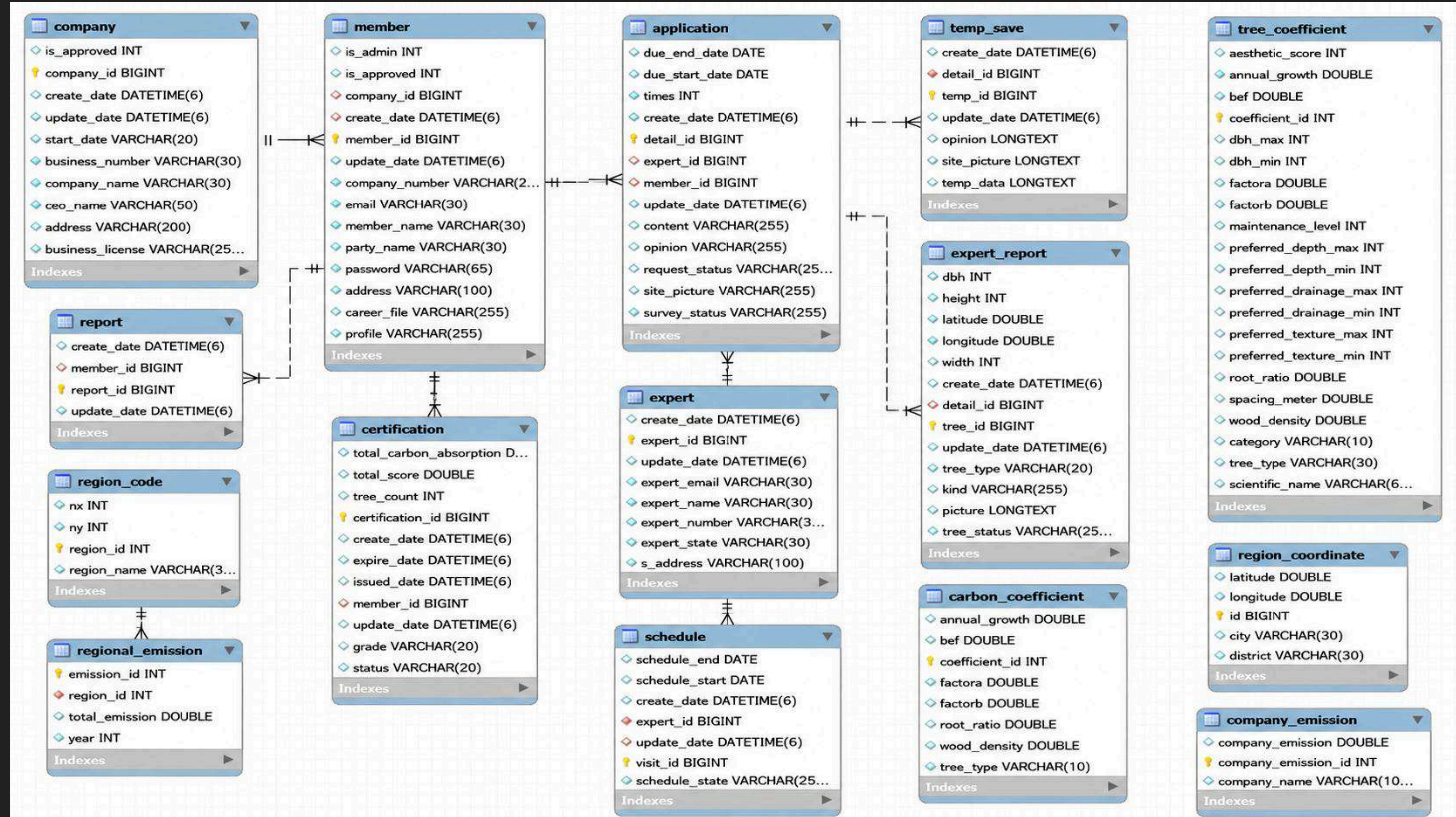


API & Database



GROOT API SPEC

QR을 스캔하면 Groot 백엔드 API 명세서로 이동합니다.



Core Logic

```
// 보정된 탄소흡수량 계산
double soilRatio = soilScore / 30.0; // 토양 보정비
double weatherRatio = weatherScore / 20.0; // 날씨 보정비
double adjustedCarbon = baseCarbonAbsorption * soilRatio * weatherRatio;

// 총점 기준 상위 3종 선정
List<TreeRecommendation> top3 = candidates.stream()
    .sorted(Comparator.comparingDouble(TreeScore::getTotal).reversed())
    .limit(3)
    .map(s -> TreeRecommendation.builder()
        .treeName(s.getTreeName())
        .totalScore(s.getTotal())
        .adjustedCarbon(s.getAdjustedCarbon()) // 보정 흡수량
        .reason(s.getReason()) // 추천 사유
        .build());
```

코드 사진

TREE RECOMMENDATION FLOW

수목 추천 알고리즘

기업 입력: 지역 · 면적 · 희망수량

↓

① 토양 API(흙토람): 배수등급, 토심, 토성 등 6가지

② 기상청 API: 실시간 기온, 습도

③ 22종 수종 DB: 상대생장식 계수 + 토양 선호도

↓

수종별 4항목 점수화

↓

상위 3종 추천 + 보정 탄소흡수량 + 추천 사유

수목 추천

지역 기후·토양 기반으로 최적 수종을 추천받아보세요.

핵심 로직 화면

1 기본 정보 — 2 수종 추천 결과

📍 서울 🌿 면적: 2,000 m² 🌱 면적 기준 최대 식재: 222그루

추천 수종 TOP 3

1 신갈나무 낙엽활엽수

★ 91.5 / 100점

그루당 연간 흡수 7.6 kg
연간 총 흡수량 229.06 kg
식재 간격 5.0 m

📖 배수조건 적합. 토심 적합. 기온 최적 구간. 예상 연간 흡수량 10.5kg/그루.

탄소흡수량 (35%)	93
토양적합도 (30%)	97
기후적합도 (20%)	75
면적적합도 (15%)	100

2 아까시나무 낙엽활엽수

★ 89.5 / 100점

그루당 연간 흡수 8.1 kg
연간 총 흡수량 243.7 kg
식재 간격 3.0 m

📖 배수조건 적합. 토심 적합. 기온 최적 구간. 예상 연간 흡수량 11.2kg/그루.

탄소흡수량 (35%)	100
토양적합도 (30%)	97
기후적합도 (20%)	75
면적적합도 (15%)	70

3 상수리나무 낙엽활엽수

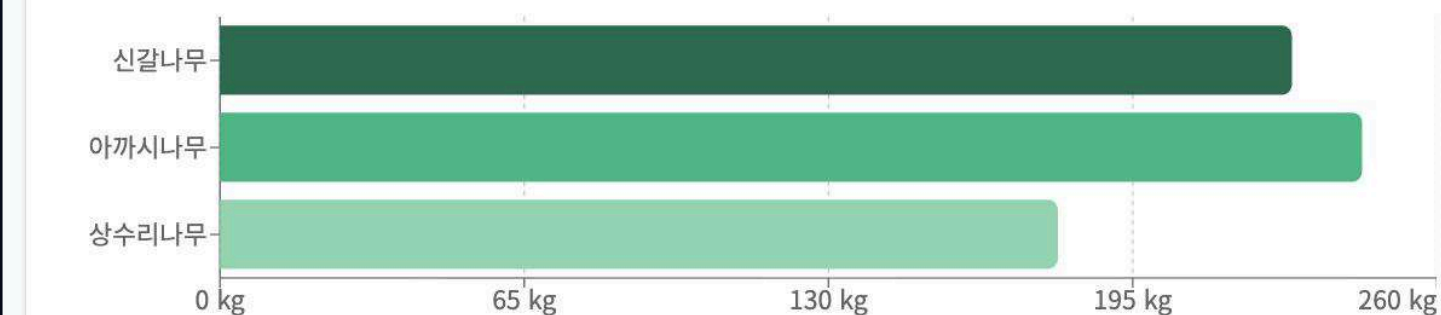
★ 82.9 / 100점

그루당 연간 흡수 6.0 kg
연간 총 흡수량 178.84 kg
식재 간격 5.0 m

📖 배수조건 적합. 토심 적합. 기온 최적 구간. 예상 연간 흡수량 8.2kg/그루.

탄소흡수량 (35%)	68
토양적합도 (30%)	97
기후적합도 (20%)	75
면적적합도 (15%)	100

수종별 연간 탄소흡수량 비교 (30그루 기준)



← 조건 다시 입력

SCORING & CO2 ESTIMATION

최종 추천

탄소흡수력(35) + 토양(30) + 날씨(20) + 면적(15) = 100점

상위 3종 추천과 보정된 탄소흡수량을 함께 제공

보정 탄소흡수량 = 기본 흡수량 × 토양 보정비 × 날씨 보정비

→ 실제 환경을 반영한 예상 CO₂ 흡수량(kg) 제공

Troubleshooting

```
// 정규화용 최대/최소 계산 → 22종 × 2번 호출
double maxCarbon = allSpecies.stream()
    .mapToDouble(this::estimateAnnualCarbon).max().orElse(1);
double minCarbon = allSpecies.stream()
    .mapToDouble(this::estimateAnnualCarbon).min().orElse(0);

for (TreeCoefficient coeff : allSpecies) {
    double carbonScore = calcCarbonScore(coeff, ...); // 내부에서 또 호출
    double carbonPerYear = estimateAnnualCarbon(coeff); // 또 호출
    // buildReason 내부 → estimateAnnualCarbon(coeff) // 또 호출
}
```

Before

```
Map<Integer, Double> carbonCache = allSpecies.stream()
    .collect(Collectors.toMap(
        TreeCoefficient::getCoefficientId,
        this::estimateAnnualCarbon));

double maxCarbon = carbonCache.values().stream()
    .mapToDouble(Double::doubleValue).max().orElse(1);
double minCarbon = carbonCache.values().stream()
    .mapToDouble(Double::doubleValue).min().orElse(0);

for (TreeCoefficient coeff : allSpecies) {
    double cachedCarbon = carbonCache.get(coeff.getCoefficientId());
    double carbonScore = calcCarbonScoreFromCache(cachedCarbon, ...);
    double carbonPerYear = cachedCarbon * soilFactor * weatherFactor;
```

After



TROUBLESHOOTING

estimateAnnualCarbon 중복 호출 제거

문제 정의

동일한 estimateAnnualCarbon() 메서드가 수종당 5번 중복 호출됨. 22종 기준 min/max 2번, 점수계산 1번, 보정 1번, 추천사유 1번으로 총 110번 반복 발생.

원인 추론

계산 결과를 저장하지 않고 필요할 때마다 매번 새로 호출하는 구조였음. 입력값인 수종별 계수가 동일하므로 결과도 항상 같지만, 캐시 없이 반복 연산이 발생함.

해결책 구현

Map<Integer, Double>에 수종별 결과를 1번만 계산해 캐싱함. 이후 모든 호출을 캐시 조회로 대체해 110번에서 22번으로 감소, 반복 연산 약 80% 절감.

110회 → 22회 · 반복 연산 약 80% 감소

Retrospective

KEEP

잘 된 것 · 계속

- ✓ 회원관리, 로직, 보고서, 탄소흡수량 산정, 인증마크 관련 작업을 역할별로 분담해 병렬 진행
- ✓ 주소 파싱, 탄소 계산, 수종 추천처럼 핵심 로직의 책임 범위를 명확히 나누어 작업 효율 개선
- ✓ 스프린트 단위로 JIRA 상태를 갱신하며 수정 이슈를 빠르게 확인하고 피드백 반영

PROBLEM

아쉬운 것 · 개선 대상

- × 자동화 알고리즘 설계 과정에서 추가 DB 설계와 수정 코드가 많아져 이전 스프린트 코드 보완 시간이 부족
- × API 문서화가 늦어 프론트엔드 연동과 기능 테스트 과정에서 혼선 발생
- × 주소 파싱, 탄소 계산 등 핵심 로직 검증이 수동 테스트에 의존해 버그 발견이 늦어지는 경우 발생

TRY

다음에 시도할 것

- 스프린트 시작 전 설계 검토 시간을 확보하고, 미흡한 코드는 백로그에 리팩토링 태스크로 등록
- Swagger 기반 API 문서화를 구현과 병행하고, 요청/응답 형식을 사전에 합의
- 주소 형식 통일, 유효성 검증, 탄소 계산 로직에 대한 단위 테스트를 작성해 데이터 정확성 사전 방지

— CLOSING

Closing.

구조를 이해하고,
데이터로 검증하며,
사용자에게 닿는 기능을 만들겠습니다.

문제를 기능으로만 보지 않고, 흐름과 책임으로 나누어 설계하는 개발자.
작게 만들고, 끝까지 확인하며, 실제 사용자의 맥락까지 연결하는 개발자로 성장하겠습니다.

BUILD with structure · VERIFY with data · CONNECT to users

BACKEND · AI FULLSTACK

SungEun Kim

kimsungeun0318@naver.com / Seoul, Korea

GITHUB : <https://github.com/Sungeun0318>

+82 10 · 6462 · 0602